

Chapter 5:

Process Scheduling

Chien Chin Chen

Department of Information Management
National Taiwan University

Outline

- Basic Concepts.
- Scheduling Criteria.
- Scheduling Algorithms.
- Multiple-Processor Scheduling.
- Thread Scheduling.
- Operating Systems Examples.
- Algorithm Evaluation.

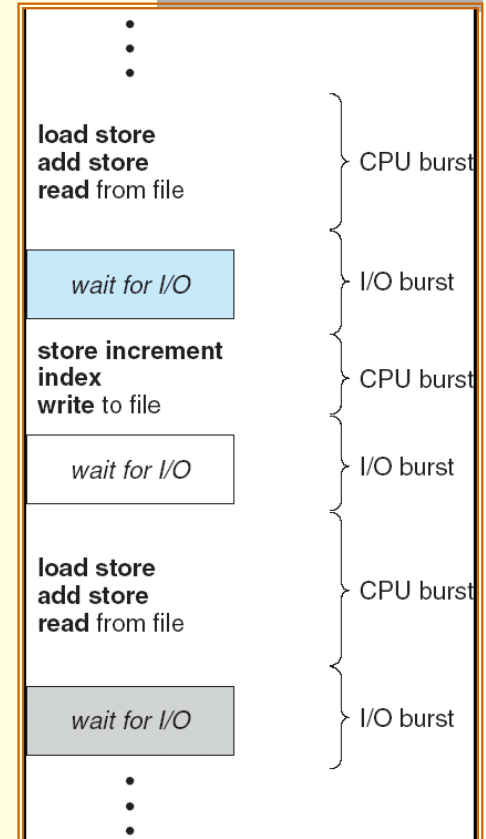
Basic Concepts (1/3)

- In a single-processor, only one process can run at a time; any others must wait until the CPU is free and can be rescheduled.
- **Multiprogramming** is to have some process running at all times, to maximize CPU utilization.
 - Several processes are kept in memory at one time.
 - A process is executed until it must wait (for example I/O).
 - When one process has to wait, the operating system **takes** the CPU **away** from that process and **gives** the CPU **to another** process.
- **Scheduling** of this kind is a fundamental operating-system function.
 - Almost all computer resources are scheduled before use.
 - Here, we talk about **CPU scheduling** as it is central to operating-system design.

★ 只要有競爭資源的地方,就需要 scheduling

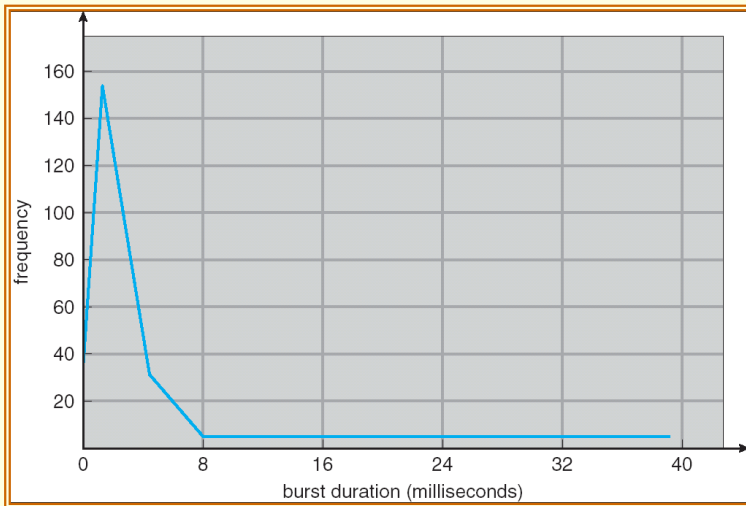
Basic Concepts (2/3)

- Process execution consists of a cycle of **CPU execution** and **I/O wait**.
 - This is, processes alternate between these two states.
- Process execution generally begins with a CPU burst, followed by an I/O burst.
 - Which is followed by another CPU burst, and so on.
 - Eventually, the final CPU burst ends with a request to terminate execution.



Basic Concepts (3/3)

- To have a successful CPU scheduler, we have to know properties of process execution.
- The figure shows the durations of CPU bursts.
 - Processes generally have a large number of short CPU bursts and small number of long CPU bursts.



- An I/O-bound program typically has many short CPU bursts.
- A CPU-bound program might have a few long CPU bursts.

CPU Scheduler (1/4)

- **CPU scheduler** (short-term scheduler) select one of the processes in the ready queue to be executed, whenever the CPU becomes idle.
- The ready queue is **not necessarily** a *first-in, first-out* (FIFO) queue.
 - With various scheduling algorithms, a ready queue can be implemented as a FIFO queue, a priority queue, a tree, or simply an unordered linked list.
- However, all the processes in the ready queue are waiting for a chance to run on the CPU.
 - The records in the queues are generally *process control blocks* (PCBs) of the processes.

CPU Scheduler (2/4)

- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state (I/O or wait for child processes).
 2. Switches from running to ready state (interrupted).
 3. Switches from waiting to ready (completion of I/O).
 4. Terminates.
- When scheduling takes place only under circumstances 1 and 4, we say that the scheduling scheme is **nonpreemptive**. (不會搶人)
 - Otherwise, it is **preemptive**.

↳ 較有效率

CPU Scheduler (3/4)

- Under nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.
 - Windows 3.x.
 - Windows 95 and all subsequent versions of Windows have used preemptive scheduling.
- Preemptive scheduling usually has better scheduling performances. However, **it incurs data inconsistency**.
 - Consider the case of two processes that share data.
 - While one is updating the data, it is preempted so that the second process can run.
 - The second process then tries to read the data, which are in an inconsistent state.
 - Chapter 6 discuss mechanism of coordination.

CPU Scheduler (4/4)

- During the processing of a system call, the operating-system kernel may be busy with an activity on behalf of a process.
 - The activity may involve changing important kernel data (e.g., I/O queues).
- If the process is preempted in the middle of these changes and the kernel (or the device driver) need to read or modify the same structure, chaos ensues.
- Therefore, certain operating systems (UNIX) deal with this problem by waiting for system call to complete before doing a context switch.

Dispatcher

- **Dispatcher**: a module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context.
 - switching to user mode.
 - jumping to the proper location in the user program to restart that program.
- The dispatcher should be *as fast as possible*, since it is invoked during every process switch.
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria (1/4)

- Different CPU scheduling algorithms have different properties, and favor one class of processes over another.
- Many criteria have been suggested for comparing CPU scheduling algorithms.
 - Different criteria are used for comparison can make a substantial difference in which algorithm is judged to be best.
- ***CPU utilization:***
 - The utilization of CPU.
 - Can range from 0 to 100 percent.
 - It should range from 40 percent (lightly loaded system) to 90 percent (heavily used system).

Scheduling Criteria (2/4)

■ **Throughput:**

- The number of processes that complete their execution per time unit.
- For example – for long processes, the rate may be one process per hour. For short processes, it may be 10 processes per second.

■ **Turnaround time:**

- Amount of time to execute a particular process.
- Is the sum of the periods spent waiting in the ready queue, executing on the CPU, doing I/O, ...

Scheduling Criteria (3/4)

■ **Waiting time:**

- Since a CPU scheduling algorithm only select processes in ready queue for execution, it does not affect the amount of time during which a process executes or does I/O.
- Therefore, the cost of an algorithm is the amount of time a process has been **waiting in the ready queue**.

■ **Response time:** (不代表結束, 只算feedback)

- In an interactive system, turnaround time may not be the best criterion.
- Often, a process can produce some output fairly early and can continue computing new results.
- This measure is the amount of time it takes from when a request was submitted until the **first response** is produced.

Scheduling Criteria (4/4)

- In most cases, the goal of a scheduling algorithm is to optimize the **average** measure (e.g., the average throughput).
 - The average comes from the results of many empirical (simulated) processes.
- But .. Investigators have suggested that, for interactive system (such as time-sharing systems), it is more important to minimize the **variance** in the response time than to minimize the average response time.
 - A system with reasonable and **predictable** response time may be considered more desirable than a system that is faster on the average but is highly variable.
- However, little work has been done on minimizing variance.

Scheduling Algorithms (1/33)

- For simplicity, we consider only one CPU burst per process in the following examples.
 - In reality, each process can have several hundred CPU bursts and I/O bursts.
- The measure of comparison is the average waiting time.

考到這

Scheduling Algorithms –

First-Come, First-Served Scheduling (2/33)

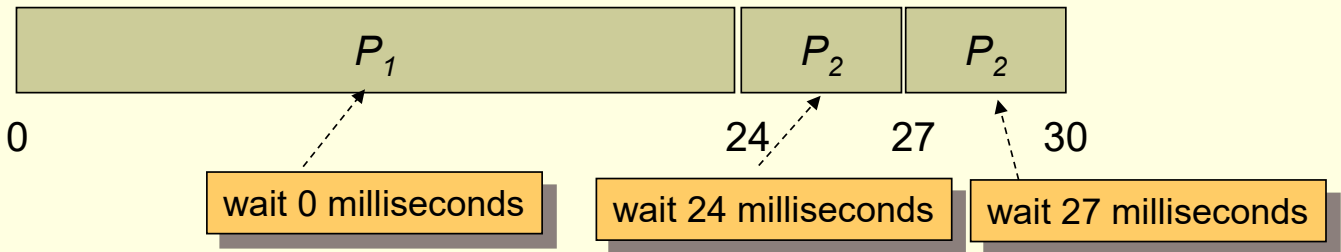
- The simplest CPU-scheduling algorithm is the ***first-come, first-served*** (FCFS) scheduling algorithm.
- The process that requests the CPU first is allocated the CPU first.
- Can be easily managed with a FIFO queue.
 - When a process enters the ready queue, its PCB is linked onto the **tail** of the queue.
 - When the CPU is free, it is allocated to the process at the **head** of the queue.

Scheduling Algorithms –

First-Come, First-Served Scheduling (3/33)

Process	Burst Time (ms.)
P_1	24
P_2	3
P_3	3

- If the process arrive in the order P_1, P_2, P_3 at time 0. Then, we get the result shown in the following Gantt chart:

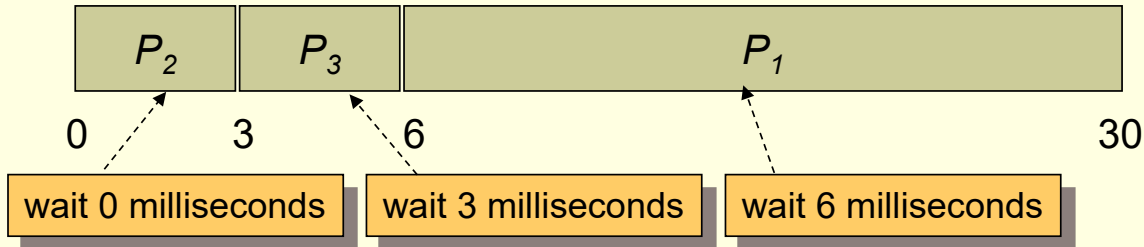


The average waiting time is $(0+24+27)/3 = 17$ milliseconds

Scheduling Algorithms –

First-Come, First-Served Scheduling (4/33)

- If the process arrive in the order P_2, P_3, P_1 , the results will be as follows:



The average waiting time is $(6+0+3)/3 = 3$ milliseconds

- The average waiting time under an FCFS policy is generally **not minimal** and **may vary substantially**.

Scheduling Algorithms –

First-Come, First-Served Scheduling (5/33)

- The FCFS scheduling algorithm is **nonpreemptive**.
 - Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU, either by terminating or by requesting I/O.
- Consider FCFS scheduling in a dynamic situation where we have one CPU-bound process and many I/O-bound process.
 - The CPU-bound process will get and hold the CPU.
 - All the other processes will finish their I/O and will move into the ready queue, waiting for the CPU. ← **The I/O devices are idle.**
 - Eventually, the CPU bound process moves to an I/O devices.
 - All the I/O-bound process execute quickly and move back to the I/O queues. ← **The CPU sits idle.**
 - Again, the CPU-bound process will then move back and hold the CPU, and all the I/O processes have to wait in the ready queue.

Scheduling Algorithms –

First-Come, First-Served Scheduling (6/33)

- The above situation is called a ***convoy effect***.
 - All the other processes wait for the one big process to get of the CPU.
 - Result in **lower CPU** and **device utilization**.

Scheduling Algorithms –

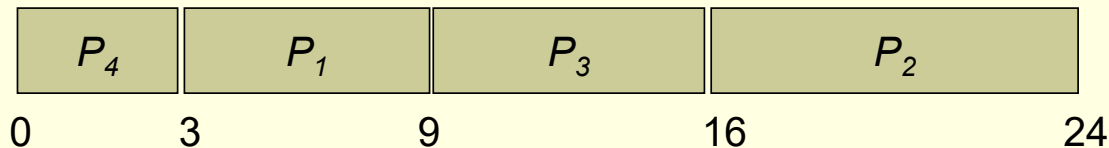
Shortest-Job-First Scheduling (7/33)

- The shortest-job-first (SJF) scheduling algorithm associates with each process the length of the process's next CPU burst.
 - Another more appropriate term – ***shortest-next-CPU-burst algorithm***.
- When the CPU is available, it is assigned to the process that has the smallest next CPU burst.
 - If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.

Scheduling Algorithms –

Shortest-Job-First Scheduling (8/33)

Process	Burst Time (ms.)
P_1	6
P_2	8
P_3	7
P_4	3



- The average waiting time is $(3+16+9+0)/4 = 7$ ms.
- By comparison, if we were using the FCFS, the average waiting time would be 10.25 ms.

Scheduling Algorithms –

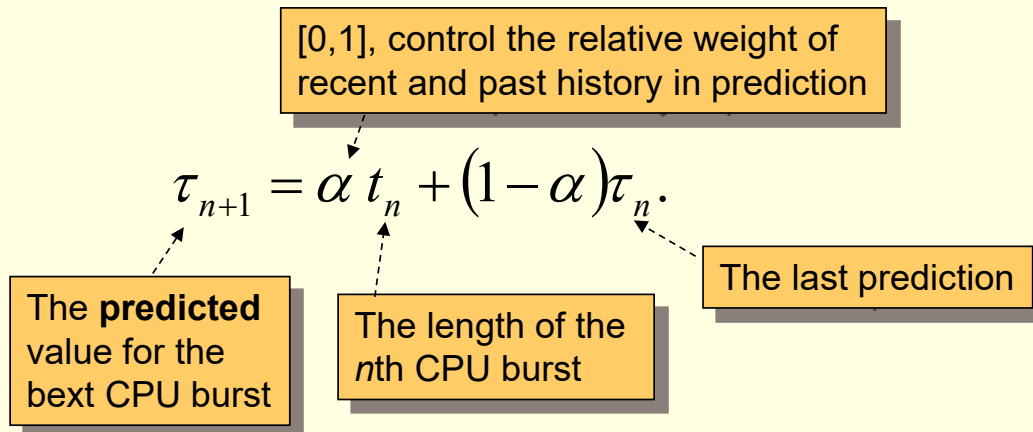
Shortest-Job-First Scheduling (9/33)

- **The SJF scheduling algorithm is optimal!!**
 - It gives the minimum average waiting time for a given set of processes.
 - Why?
 - Moving a short process before a long one decreases the waiting time of the short process more than it increases the waiting time of the long process.
- But ... how can we obtain the length of the next CPU burst??
 - **It is impossible to do so ...**
- However, we can try our best to **predict** the length.
 - We expect that the next CPU burst will be similar in length to the previous ones.
 - And pick the process with the shortest predicted CPU burst.

Scheduling Algorithms –

Shortest-Job-First Scheduling (10/33)

- **Prediction** – the *exponential average* of the measured lengths of previous CPU bursts.



- If $\alpha=1$, then only the most recent CPU burst matters.
- More commonly $\alpha=0.5$; recent history and past history are equally weighted.

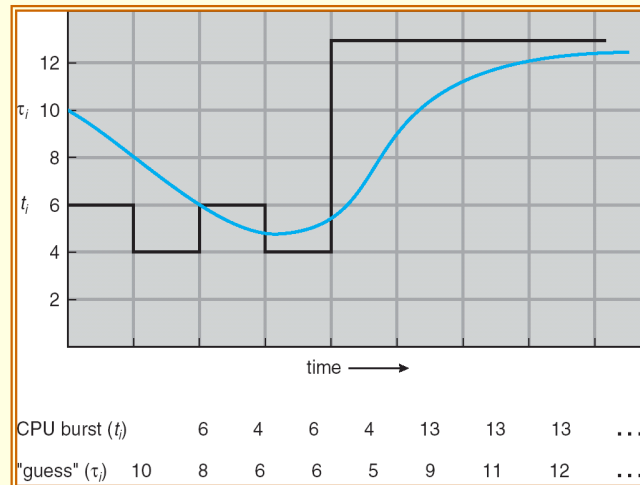
Scheduling Algorithms –

Shortest-Job-First Scheduling (11/33)

- We can expand the formula to find:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\alpha t_{n-1} + (1-\alpha)^2 \alpha t_{n-2} + \dots + (1-\alpha)^j \alpha t_{n-j} + \dots + (1-\alpha)^{n+1} \tau_0$$

- τ_0 can be defined as a constant or as an overall system average.
- Since both α and $(1-\alpha)$ are **less than or equal to 1**, each successive term has less weight than its predecessor.



Scheduling Algorithms –

Shortest-Job-First Scheduling (12/33)

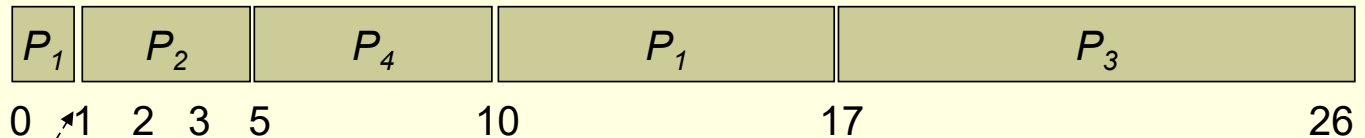
- The SJF algorithm can be either **preemptive** or **nonpreemptive**.
 - A preemptive SJF algorithm will preempt the currently executing process, if the next CPU burst of the newly arrived process is shorter than “what is left” of the currently executing process.
 - Whereas a nonpreemptive SJF algorithm will allow the currently running process to finish its CPU burst.

Scheduling Algorithms –

Shortest-Job-First Scheduling (13/33)

- An example of **preemptive** SJF algorithm:

Process	Arrival Time	(remaining) Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5



Preempted!!

This algorithm is also known as
shortest-remaining-time-first scheduling

Scheduling Algorithms –

Shortest-Job-First Scheduling (14/33)

- The average waiting time for preemptive algorithm is $((10-1) + (1-1) + (17-2) + (5-3)) / 4 = 6.5$ ms.
- The average waiting time for nonpreemptive algorithm is 7.75ms.
- SJF scheduling is used frequently in **long-term scheduling**, where users need to specify “**process time limit**” of the processes.
 - The algorithm encourages users have accurate estimations because:
 - A lower value mean faster response.
 - And too low a value will cause a time-limit-exceeded error and require resubmission.

Scheduling Algorithms –

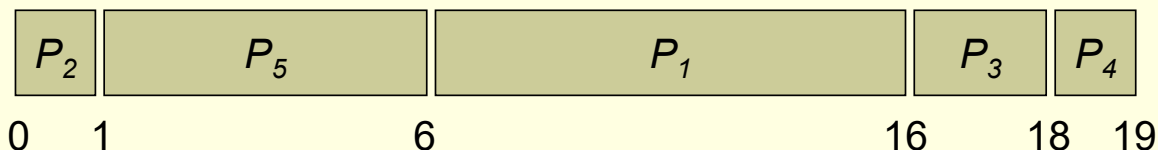
Priority Scheduling (15/33)

- A priority is associated with each process, and the CPU is allocated to the process with the highest priority.
 - Equal-priority processes are scheduled in FCFS order.
 - The SJF algorithm is a special case of the priority scheduling algorithm.
 - The larger the CPU burst, the lower the priority, and vice versa.
- Priorities are generally indicated by some fixed range of numbers.
 - In this text, we assume that low numbers represent high priority.

Scheduling Algorithms – Priority Scheduling (16/33)

Process	Burst Time	Priority
● P_1	10	3
● P_2	1	1
● P_3	2	4
● P_4	1	5
● P_5	5	2

- If the process arrive in the order $P_1, P_2, \dots, P_5$ at time 0, we would schedule these processes as follows:



- The average waiting time is 8.2 ms.

Scheduling Algorithms –

Priority Scheduling (17/33)

- Priorities can be defined either **internally** or **externally**.
 - Internally – use measurable quantity in terms of time limits, memory requirements ...
 - Externally – set by criteria outside the operating system, such as importance, funds, ... often political factors.
- Priority scheduling can be either **preemptive** or **nonpreemptive**.
 - Preemptive – can preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.
 - Nonpreemptive – simply put the new process at the head of the ready queue.

Scheduling Algorithms –

Priority Scheduling (18/33)

■ **Starvation:**

- **Low-priority processes can be blocked indefinitely.**
- A major problem with priority scheduling algorithms.
- May occur in a heavily loaded computer system with steady stream of higher-priority processes.
- *Rumor: when the IBM 7094 at MIT shut down in 1973, administrators found a low-priority process that had been submitted in 1967, and had not yet been run.*
- Solution: **aging**.
 - Gradually increase the priority of processes that wait in the system for a long time.
 - E.g., by 1 every 15 minutes.
 - A process would eventually arrive the highest priority and would be executed.

Scheduling Algorithms –

Round-Robin Scheduling (19/33)

- Is designed especially for time sharing systems.
- Is similar to FCFS scheduling, but **preemption** is added to switch between processes.
- The ready queue is treated as a **circular queue**.
 - New processes are added to the tail of the ready queue.
- A small unit of time, called a **time quantum** or **time slice**, is defined.
 - Generally 10 to 100 ms.
 - A timer is interrupted after 1 time quantum.
 - The scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

Scheduling Algorithms –

Round-Robin Scheduling (20/33)

- If the CPU burst of the selected process is less than 1 time quantum ...
 - The process itself will release the CPU.
 - The scheduler will then proceed to the next process in the ready queue.

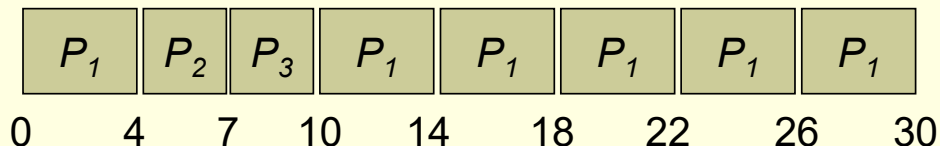
- If the CPU burst is longer than 1 time quantum ...
 - The timer will go off and will cause an interrupt to create context switch.
 - **That is, the process is preempted.**
 - The process will be put at the tail of the ready queue.
 - The scheduler will then select the next process in the ready queue.

Scheduling Algorithms –

Round-Robin Scheduling (21/33)

Process	Burst Time (ms.)
P_1	24
P_2	3
P_3	3

- If the process arrive in the order P_1, P_2, P_3 at time 0 and a time quantum is 4 ms. Then the process schedule is:



- The average waiting time is $(6 + 4 + 7) / 3 = 5.66$ ms.

Scheduling Algorithms –

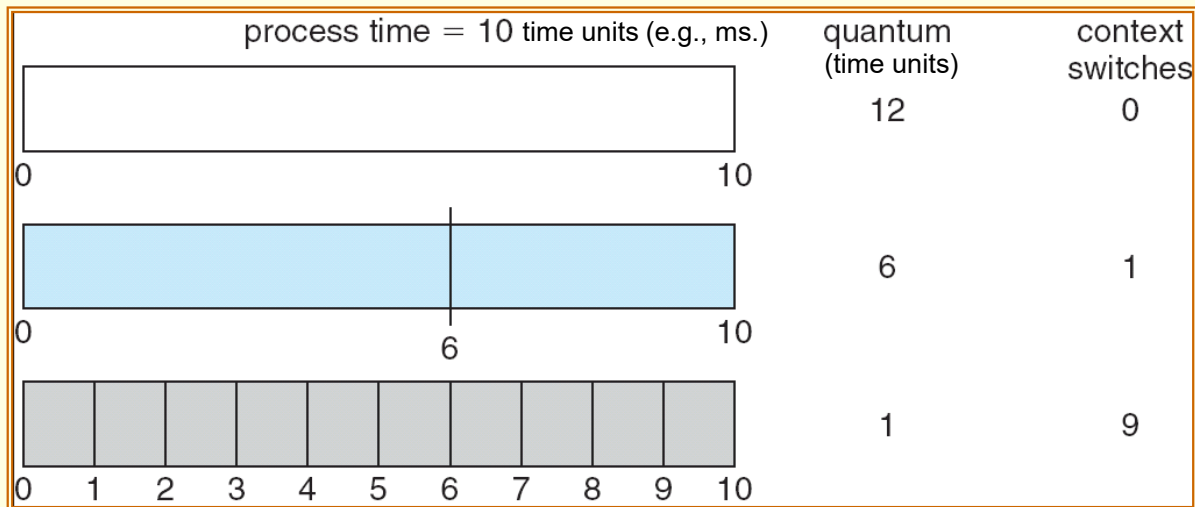
Round-Robin Scheduling (22/33)

- If there are n processes in the ready queue and the time quantum is q ,
 - Each process must wait **no longer** than $(n-1) \times q$ time units until its next time quantum.
- The average waiting time under the RR policy is often long.
 - And the waiting time is proportional to the number of processes.
 - But the waiting is used to trade for time sharing phenomenon.
- The performance of the RR algorithm depends heavily on the size of the time quantum.
 - If the time quantum is extremely large, the RR policy is the same as the FCFS policy.
 - If the time quantum is extremely small (say, 1 ms), the RR approach is called processor sharing and (in theory) creates the appearance that each of n processes has its own processors running at $1/n$ the speed of the real processor.

Scheduling Algorithms –

Round-Robin Scheduling (23/33)

- But ... do not forget the effect of context switching.

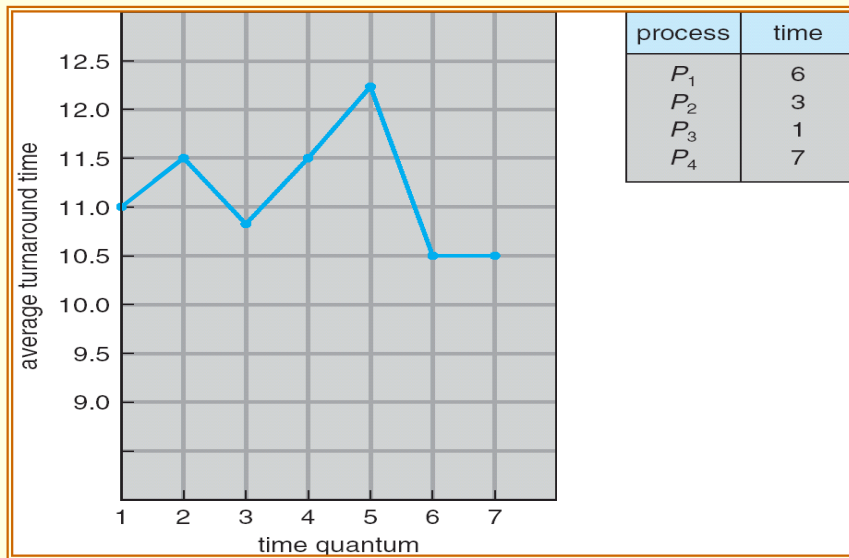


- We want the time quantum to be large with respect to the context switch time.
 - In practice, operating systems have time quanta ranging from 10 to 100 milliseconds (10^{-3}).
 - The time of context switch is typically less than 10 microseconds (10^{-6}).
 - Only a small fraction of the CPU time spent in context switching.

Scheduling Algorithms –

Round-Robin Scheduling (24/33)

- The average turnaround time of a set of processes **does not** necessarily improve as the time-quantum size increases.



Scheduling Algorithms –

Round-Robin Scheduling (25/33)

- In general, the average turnaround time can be improved if most processes finish their **next** CPU burst in a single time quantum.
 - This implies the time quantum should be long.
 - For example, given 3 processes of 10 time units.
 - The average turnaround time is 29 for a quantum of 1 time unit.
 - However, the average turnaround time drops to 20 if the time quantum is 10.
 - Moreover, if the cost of context-switch is added in, the average turnaround time **increases for a smaller time quantum.**
- But remember that ... the time quantum should not be too large.
 - Otherwise RR scheduling degenerates to FCFS policy.
 - A rule of thumb is that 80 percent of the CPU bursts should be shorter than the time quantum.

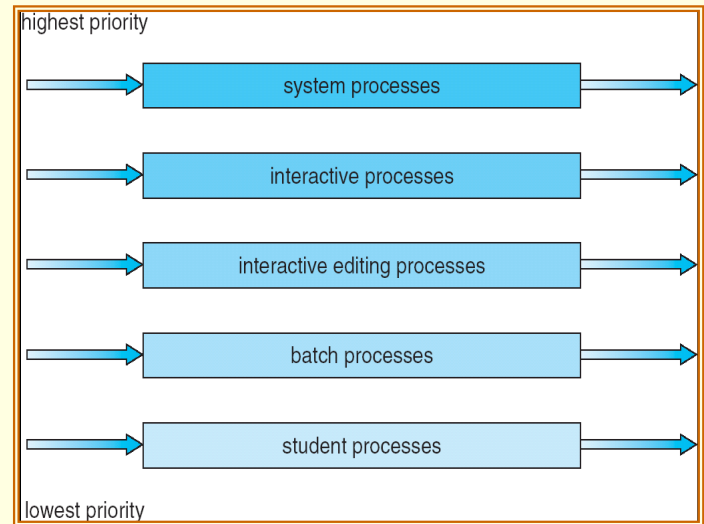
Scheduling Algorithms –

Multilevel Queue Scheduling (26/33)

- Processes are classified into different groups.
 - Different types of processes require different response-time and have different scheduling needs.
 - For examples, foreground (interactive) and background (batch) processes.
- The scheduling algorithm partitions the ready queue into several separate queues.
 - The processes are assigned to one queue, based on some property of the process (memory size, process priority ...).
 - Each queue has its own scheduling algorithm.
 - The foreground queue might be scheduled by an RR algorithm.
 - The background queue is scheduled by an FCFS algorithm.
 - Moreover, there must be scheduling among the queues.
 - Usually a fixed-priority preemptive scheduling.

Scheduling Algorithms – Multilevel Queue Scheduling (27/33)

- No process in batch queue could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty.
- If an interactive editing process entered the ready queue while a batch process was running, the batch process would be preempted.



Scheduling Algorithms –

Multilevel Queue Scheduling (28/33)

- Another possibility is to time-slice among the queues.
 - Each queue gets a certain portion of the CPU time.
 - Which it can then schedule among its various processes.
 - For instance, in the foreground-background queue example:
 - The foreground queue can be given 80 percent of the CPU time for RR scheduling among its processes.
 - The background queue receives 20 percent of the CPU to give to its processes on an FCFS basis.

Scheduling Algorithms –

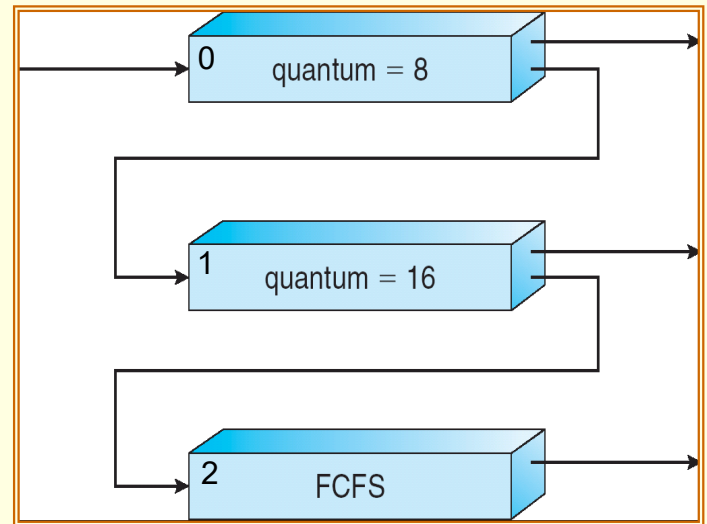
Multilevel Feedback-Queue Scheduling (29/33)

- The scheduling algorithm allows a process to move between queues.
- The idea:
 - If a process uses too much CPU time, it will be moved to a lower-priority queue.
 - A process that waits too long in a lower-priority queue may be moved to a higher-priority queue.

Scheduling Algorithms –

Multilevel Feedback-Queue Scheduling (30/33)

- In this example, the scheduler first executes all processes in queue 0.
- Only when queue 0 is empty will it execute processes in queue 1.
 - Similarly, processes in queue 2 will only be executed if queues 0 and 1 are empty.
- A process in queue 1 will in turn be preempted by a process arriving for queue 0.



Scheduling Algorithms –

Multilevel Feedback-Queue Scheduling (31/33)

- A process (formally, a CPU burst) entering the ready queue is put in queue 0.
 - It will be given a time quantum of 8 ms.
 - If it (the burst) does not finish within this time, it is moved to the tail of queue 1.
- If queue 0 is empty, the process at the head of queue 1 is given a quantum of 16 ms.
 - If it (the burst) does not complete, it is preempted and is put into queue 2.
- Processes in queue 2 are run on an FCFS basis.
 - But are run only when queues 0 and 1 are empty.

Scheduling Algorithms –

Multilevel Feedback-Queue Scheduling (32/33)

- This example gives highest priority to processes with CPU burst^s of 8 milliseconds or less.
 - Such a process will quickly get the CPU.
 - Finish its CPU burst.
 - And go off to its next I/O burst.
- Processes (CPU bursts) that need more than 8 but less than 24 milliseconds are also served quickly.
- Long processes automatically sink to queue 2 and are served in FCFS order with any CPU cycles left over from queues 0 and 1.

Scheduling Algorithms –

Multilevel Feedback-Queue Scheduling (33/33)

- A multilevel-feedback-queue scheduler is generally defined by the following parameters:
 - Number of queues.
 - Scheduling algorithms for each queue.
 - Method used to determine when to upgrade a process.
 - Method used to determine when to demote a process.
 - Method used to determine which queue a process will enter when that process needs service.
- The flexibility makes it the most general CPU-scheduling algorithm.
 - Can be configured to match a specific system.
 - Unfortunately, it is also the most complex algorithm (parameter setting).

Multiple-Processor Scheduling (1/5)

■ *Asymmetric multiprocessing*:

- Has all scheduling decision, I/O processing, and other system activities handled by a single processor – **the master server**.

■ *Symmetric multiprocessing* (SMP):

- Each processor is self-scheduling.
- All processes may be in a common ready queue, or each processor may have its own private queue of ready processes.
- Scheduling schedules each processor to examine the ready queue and selects a process to execute.
- Virtually all modern operating systems support SMP.

Multiple-Processor Scheduling (2/5)

- **Cache** in storage hierarchy:
 - The data most recently accessed by a process populates the cache for a processor.
 - Successive memory access by the process are often satisfied in cache memory.
- If the process migrates to another processor ...
 - The contents of cache memory must be **invalidated** for the processor being migrated from.
 - The cache for the processor being migrated to must be **re-populated**.
- To avoid the cost, most SMP systems try to avoid migration of processes between processors.
 - This is known as **processor affinity**, meaning that a process has an affinity for the processor on which it is currently running.
 - **Soft affinity**: try to, but not guarantee that it will do so.
 - **Hard affinity**: provide system calls for the forbiddance.

Multiple-Processor Scheduling (3/5)

■ ***Load balancing:***

- Keep the workload balanced among all processors.
- Fully utilize the benefits of multi-processors.
- Is often **unnecessary** on systems with a common ready queue.
 - Once a processor becomes idle, it immediately extracts a runnable process from the common ready queue.
 - However, in most SMP systems, each processor does have a private queue of eligible processes.

Multiple-Processor Scheduling (4/5)

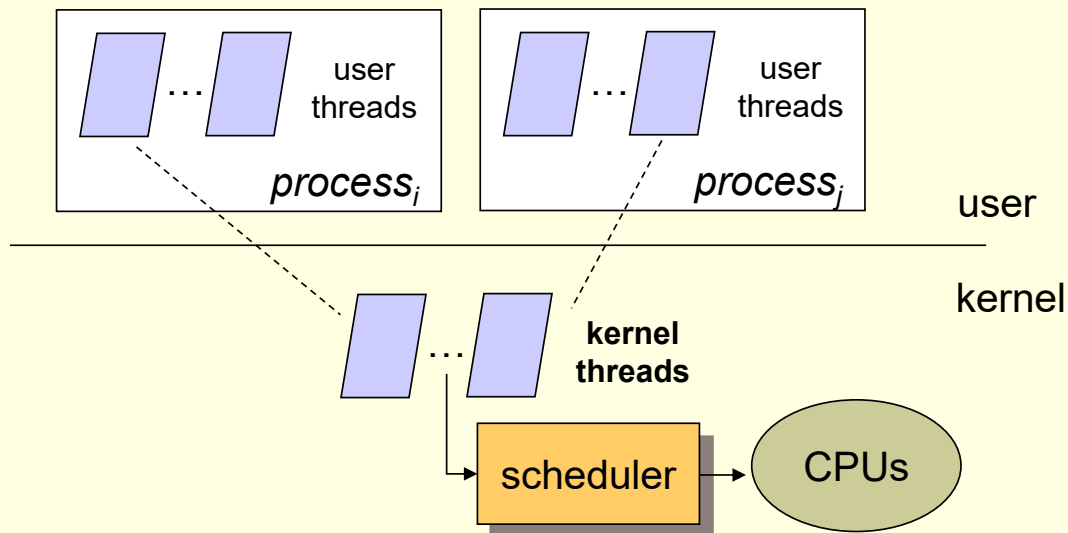
- Two general approaches to load balancing:
 - ***Push migration:***
 - A specific task (kernel process) periodically checks the load on each processor.
 - Evenly distributes the load by moving (or **pushing**) processes from overloaded to idle or less-busy processor.
 - ***Pull migration:***
 - An idle processor pulls a waiting task from a busy processor.
- Push and pull need not be mutually exclusive.
 - Linux runs its load-balancing algorithm every 200 ms (push) or whenever the run queue for a process is empty (pull).

Multiple-Processor Scheduling (5/5)

- Interestingly, load balancing often counteracts the benefits of processor affinity.
 - Either pulling or pushing a process from one processor to another will invalidate the benefit of cache memory.
 - Some systems only move processes if the imbalance exceeds a certain threshold.
- In practice, there is no absolute rule concerning what policy is best.

Thread Scheduling (1/2)

- On a **multithreaded** system:
 - User-level threads must ultimately be mapped to an associated kernel-level thread, to run on a CPU.
 - It is **kernel-level threads** that are being scheduled by the operating system.



Thread Scheduling (2/2)

- For **many-to-one** and **many-to-many** models:
 - The kernel is **unaware** of user level threads.
 - User-level threads are **managed by a thread library**.
 - The library schedules user-level threads to run on an available LWP.
 - The competition is known as **process-contention scope** (PCS), since competition for the CPU takes place among threads belonging to the same process.
 - When the thread library schedules user threads onto available LWPs, **it does not mean that the thread is actually running on a CPU.**
 - The operating system schedules the kernel thread onto a physical CPU.
 - **System-contention scope** (SCS) – kernel threads compete with each other for CPU time.
- Systems using the **one-to-one model** (such as Windows XP) schedule threads using only SCS.

Operating System Examples – Solaris (1/13)

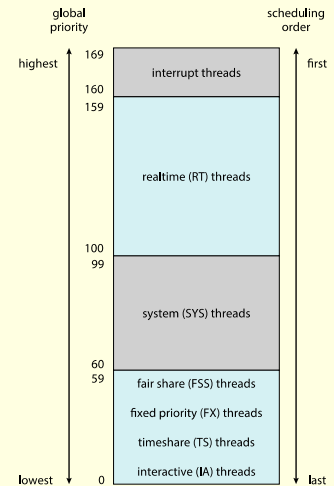
■ Solaris uses **priority-based** thread scheduling:

■ Six classes of scheduling:

- Real time.
- System.
- Fair share.
- Fix Priority
- Time sharing (default class for a process).
- Interactive (e.g., windowing applications).

■ Each class includes a set of priorities.

■ The scheduler converts the class-specific priorities into global priorities.



■ The thread with the highest global priority is selected to run.

- If there are multiple threads with the same priority, the scheduler uses a **round-robin queue**.
- Thus, the selected thread runs until it (1) blocks, (2) uses its time slice, or (3) is **preempted** by a higher-priority thread.

Operating System Examples – Solaris (2/13)

- Threads in the real-time class are given the highest priority.
 - A real-time process will run before a process in any other class.
 - In general, **few** processes belong to the real-time class.
- Kernel processes (such as the scheduler) are run in the system class.
 - The priority of a system process does not change, once established.

Operating System Examples – Solaris (3/13)

- The fixed-priority and fair-share classes were introduced with Solaris 9.
 - Threads in the fixed-priority class have the same priority range as those in the time-sharing classes.
 - However, their priorities are not dynamically adjusted.

Operating System Examples – Solaris (4/13)

- The scheduling policy for time sharing (and interactive) class **dynamically alters priorities** of threads using a **multilevel feed back queue**.

There is an **inverse** relationship between priorities and time slices.

CPU-bound processes would have lower priorities.

Interactive processes would have good response time.

priority	time quantum	time quantum expired	return from sleep
0 (low)	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59 (high)	20	49	59

The new priority of a thread that has used its entire time slice.

Penalize CPU-bound processes.

The priority of a thread that is returning from sleeping (e.g., I/O).

Provide good response time for interactive processes

Operating System Examples – Windows (5/13)

- Windows scheduler is a **priority-based, preemptive** scheduling algorithm.

- Priority classes:

- REALTIME_PRIORITY_CLASS
- HIGH_PRIORITY_CLASS
- ABOVE_NORMAL_PRIORITY_CLASS
- NORMAL_PRIORITY_CLASS
- BELOW_NORMAL_PRIORITY_CLASS
- IDLE_PRIORITY_CLASS

- Within each class is a relative priority:

- TIME_CRITICAL
- HIGHEST
- ABOVE_NORMAL
- NORMAL
- BELOW_NORMAL
- LOWEST
- IDLE

	real time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

The priority of each thread (process) is based on the priority class it belongs to and its relative priority within that class

Base priority for each class

Fixed, once determined

Operating System Examples – Windows (6/13)

- The Windows scheduler (also called dispatcher) :
 - Uses a queue **for each** scheduling priority (32-level).
 - Traverses the set of queues from highest to lowest until it finds a thread that is ready to run.

- The highest-priority thread will always run ... until:
 - It is preempted by a higher-priority thread.
 - It terminates.
 - It calls a blocking system call (such as for I/O).
 - Its time quantum ends.

Operating System Examples – Windows (7/13)

- Processes are typically members of the `NORMAL_PRIORITY_CLASS`.
 - **You can specify the class when you create a process.**
 - The initial priority of a thread is typically the **base priority** of the process the thread belongs to.
- When a thread's time quantum runs out ... and it is in the variable-priority class.
 - Its priority is lowered.
 - To limit the CPU consumption of compute-bound thread.
- When a variable-priority thread is released from a wait operation ...
 - Its priority is boosted.
 - Tend to give good response times to interactive threads.
 - I/O waiting for mouse and keyboard.

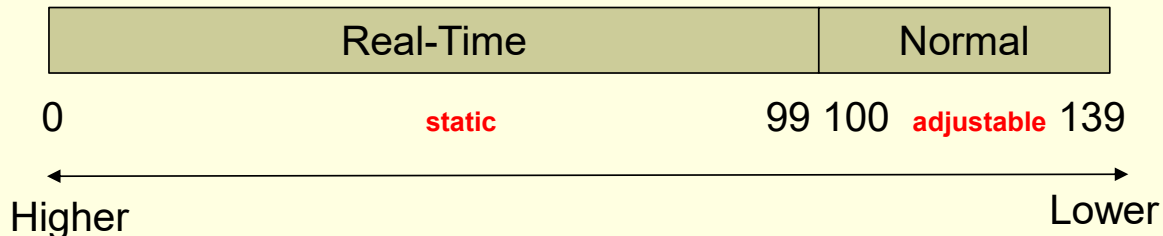
This strategy (multilevel-feedback) is used by many time-sharing operating system.

Operating System Examples – Windows (8/13)

- To provide especially good performance for a running interactive program ...
 - Windows distinguishes between the foreground process (selected on the screen) and the background processes.
 - The scheduling quantum of the foreground process is increased by some factor to give it longer time to run.
- This special scheduling rule only for process in the `NORMAL_PRIORITY_CLASS`.

Operating System Examples – **Linux** (9/13)

- In release 2.6.23 of the kernel, the **Completely Fair Scheduler** (CFS) became the default Linux scheduling algorithm.
- The Linux scheduler is based on **scheduling classes**, and is a **preemptive, priority-based** algorithm.
 - Standard Linux kernels implement two scheduling classes: (1) a default (normal) class and (2) a real-time class.



Operating System Examples – Linux (10/13)

- For the normal class:
 - Each task is associated with a ***nice value***.
 - Nice values range from -20 to +19 (0 as default).
 - Generally, a numerically lower nice value indicates a higher relative priority.
 - Unlike Solaris, tasks with lower nice values (higher priorities) receive a higher proportion CPU processing time.

Operating System Examples – Linux (11/13)

- CFS does not use discrete values of time slices!!
 - It identifies a **targeted latency**, which is an interval of time during which every runnable task should run at least once.
 - The targeted latency has default and minimum values, and can increase if the number of tasks is growing.
- Proportions of CPU time are allocated from the value of targeted latency.

Operating System Examples – Linux (12/13)

- CFS does not directly assign priorities.
 - It records how long each task **has run** using variable ***vruntime*** (virtual run time).
 - The virtual run time is associated with a decay factor based on the priority (nice value) of a task.
 - For tasks at normal priority (nice values of 0), *vruntime* is identical to actual physical run time.
 - If a higher-priority task runs for 200 milliseconds, its *vruntime* will be less than 200 milliseconds.
- To decide which task to run next:
 - CFS simply selects the task that **has the smallest *vruntime* value!!**
 - In addition, a higher-priority task the becomes available to run can **preempt a lower-priority task.**

Operating System Examples – Linux (13/13)

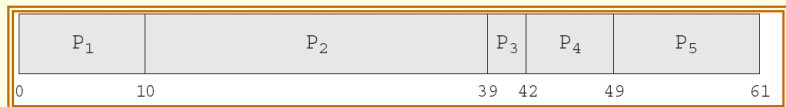
- Assume that two tasks have the same nice values.
 - One task is I/O bound and the other is CPU-bound.
- The I/O-bound task will run only for short periods.
 - The value of *vruntime* will eventually be lower for the I/O-bound task than for the CPU-bound task.
 - **This gives the I/O-bound task higher priority!!**
- If the CPU-bound task is executing when the I/O-bound task becomes eligible to run...
 - The I/O-bound task will **preempt** the CPU-bound task!!

Algorithm Evaluation – Deterministic Modeling (1/9)

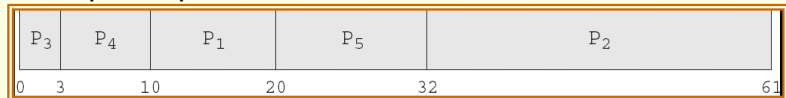
- The evaluation method takes a particular predetermined workload and defines the performance of each algorithm for that workload.
- For example, given the following workload where all processes arrive at time 0 in the order given, which algorithm would give the minimum average waiting time?

Process	Burst Time (ms.)
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

FCFS – 28ms



Nonpreemptive SJF – 13ms



RR (quantum=10ms) – 23ms



Algorithm Evaluation – Deterministic Modeling (2/9)

- Deterministic modeling is simple and fast.
- It also gives us exact numbers, allowing us to compare the algorithms.
- **However**, it requires exact numbers for input (predetermined workload), and its answers apply only to those cases.
 - On many systems the processes vary from day to day, so there is no static set of processes to use for deterministic modeling.
 - Therefore, the main uses of this method are in describing scheduling algorithms and providing examples.

Algorithm Evaluation – Queueing Models (3/9)

- The computer system is described as network of servers.
 - Each server has a queue of waiting processes.
 - The CPU is a server with its ready queue, as is the I/O system with its device queue.
 - **Queueing-network analysis**: knowing arrival rates and service rates, we can compute utilization, average waiting time ...
 - If the system is in a **steady state** — the number of processes leaving the queue must be equal to the number of processes that arrive:
 - **Little's formula**: $n = \lambda \times W$

等
的
二

The average
queue length

The average arrival rate for new process
in the queue (e.g., 3 processes/second)

The average waiting time
in the queue

Algorithm Evaluation – Queueing Models (4/9)

- We can use Little's formula to compute one of the three variables, if we know the other two.
 - If we know that 7 processes arrive every second (on average), and there are normally 14 processes in the queue,
 - Then we can compute the average waiting time per process as 2 seconds.
- Queueing analysis can be useful in comparing scheduling algorithms.
 - However, the mathematics of complicated algorithms and distributions can be difficult to work with.
 - Thus the distributions are often defined in **unrealistic** ways and may not be accurate.
 - Therefore, the accuracy of the computed results may be questionable.

Algorithm Evaluation – **Simulations** (5/9)

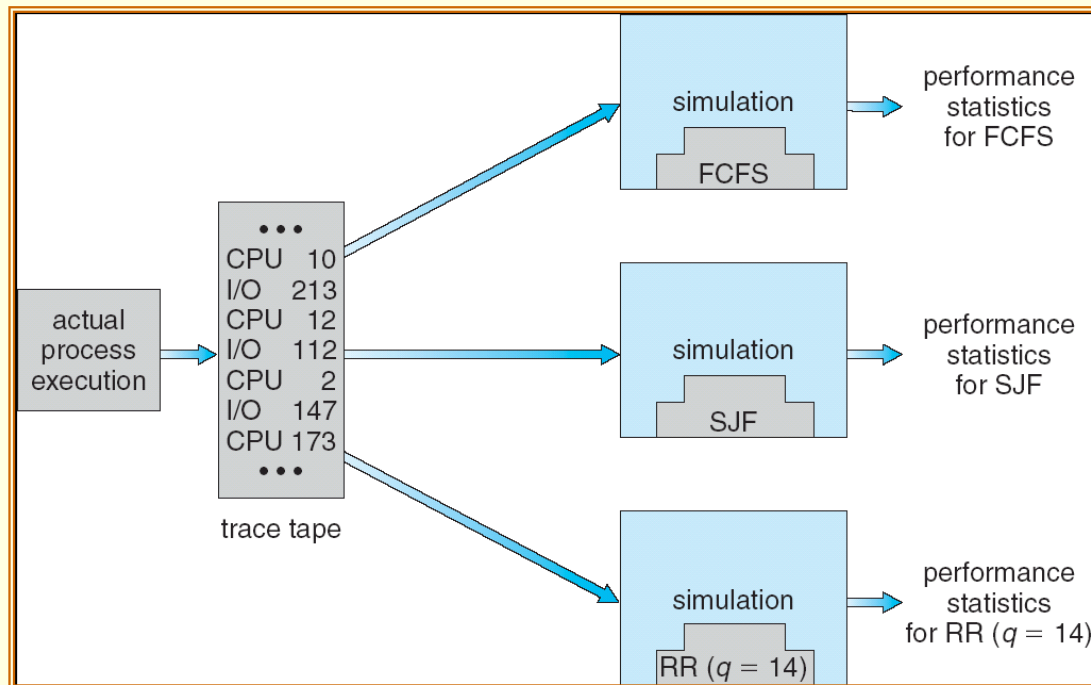
- Simulation involves:
 - Programming a model of the computer system.
 - The programmed system has a *variable* representing a *clock*.
 - As the variable's value is increased, the simulator modifies the **system state** to reflect the activities of the devices, the processes, and the scheduler.
- As the simulation executes, statistics that indicate algorithm performance are gathered and printed.
 - Usually, the evaluation result is more accurate than that of queueing models.

Algorithm Evaluation – Simulations (6/9)

- Simulators require data to drive simulations.
 - The most common method of data generation uses a **random-number generator**.
 - Which generate processes, CPU burst times, arrivals ... according to probability distributions.
 - However, a distribution-driven simulation may be inaccurate.
 - Generally, the frequency distribution indicates only how many instances of each event occur; rarely indicate the order of their occurrence.
 - To acquire more accurate result, we can use **trace tapes**.
 - A trace tape is created by monitoring the real system and recording the sequence of actual events.
 - The sequence is then used to drive the simulation.

Algorithm Evaluation – Simulations (7/9)

- Trace tapes provide an excellent way to compare different algorithms on exactly the same set of **real** inputs.



Algorithm Evaluation – Simulations (8/9)

- While simulations can acquire accurate results.
 - The process can be **expensive**, often requiring hours of computer time.
 - Trace tapes can require large amount of storage space.
 - The design, coding, and debugging of the simulator can be a difficult task.

Algorithm Evaluation – **Implementation** (9/9)

- The completely accurate way to evaluate a scheduling algorithm is to **code it up!!**
 - Put it in the operating system, and see how it works.
- The difficult with this approach is the high cost.
 - Coding the algorithm.
 - Modifying the operating system to support it.
 - **The reaction of the users to a constantly changing operating system.**



End of Chapter 5